

Animated 3D Farm Scene with Lighting and Transformations

A source-verified design, implementation, mathematics, interaction, and evaluation report



Project	3D_farm_OpenGL
Student	Niloy Chowdhury
Roll	2107117
Technology	C++17, GLFW, OpenGL 2.1 compatibility rendering
Report basis	Current source tree, successful Release build, automated tests, and direct runtime inspection

Prepared from the repository state reviewed on 10 October 2026. No institution, course, or supervisor is stated because the repository does not supply those details.

Contents and document map

Page	Section	Purpose
1	Cover	Project identity and evidence basis
2	Contents and document map	Navigation and reading guide
3	Project scope and verification	Objectives, toolchain, build and test status
4	Architecture and frame pipeline	Ownership, control flow, and active modules
5	World composition	Spatial layout, day/night overview, scene inventory
6	Farm construction	Farmhouse, crops, fences, trees, windmills, variation
7	Road and vehicles	Manual tractor, traffic convoy, driver LOD and utilities
8	Animals and field work	Cows, poultry, farmers and agricultural animation
9	Village life and shops	Three shop variants, seating, props and villagers
10	Barn system	Shell, accessible interior and detailed contents
11	Barn interaction and power	Doors, collision, props, lighting and load shedding
12	Ponds, water and bridge	Water layers, fauna, banks and Bezier bridge
13	Electrical infrastructure	Substation, poles, local lights and outage behavior
14	Sky and environmental effects	Sun, moon, clouds, birds, fog and bonfires
15	Rendering and lighting	Fixed-function pipeline, materials, transparency, lights
16	Transformation and camera mathematics	Matrices, projection, view basis and frustum culling
17	Simulation mathematics	Animation, collision, procedural placement and curves
18	Complete controls	Keyboard/mouse reference and interaction matrix
19	Performance, tests and limitations	Optimizations, measured result, validation and caveats
20	Conclusion and references	Outcomes, viva points and repository sources

Reading convention. “Active” means reachable from the current **Application/Scene** execution path. “Compiled but inactive” means present in the target but not instantiated or called by that path. Runtime screenshots are genuine captures of the built executable; no image has been illustrated or reconstructed.

Project at a glance

12farm instances	4outer barns	3ponds	5bonfire sites
3road tractors	3shop variants	7maximum ranked local lights	1automated test executable

Project scope and verification

1. Purpose and objectives

The project is an explorable low-polygon rural world designed to demonstrate hierarchical modeling, reusable geometry, interactive navigation, frame-rate-independent animation, lighting, transparency, collision, and scene-scale optimization. Its current result is substantially larger than a single model: a central road connects repeated but varied farms, village amenities, ponds, forests, barns, traffic, people, animals, and an electrical distribution site.

The implementation goal is educational clarity. Objects are built procedurally from cubes, spheres, cylinders, planes, and line-like struts; transformations remain visible in the source; animation state is updated separately from rendering; and controls expose scene systems directly. This makes the program suitable for demonstrating both OpenGL fundamentals and the practical problems that arise in a populated world.

2. Evidence and method

This report was produced by reading the complete current `include/`, `src/`, `tests/`, build configuration and README; configuring and compiling the Release target; running the automated test; launching the graphical application; exercising state toggles; and capturing the client area. Claims are tied to the current code rather than old phase descriptions.

Item	Verified result
Language and build	C++17; CMake minimum 3.16; MinGW Makefiles; MSYS2 UCRT64 GCC 16.2.0.
Libraries	GLFW 3.5.1 configuration package and Windows opengl32 through OpenGL::GL.
Graphics context	OpenGL 2.1 request using compatibility-profile fixed-function features.
Build status	Release configure and build completed successfully.
Automated test	ctest: 1/1 CameraControlTests passed in approximately 0.09 s.
Window behavior	Windowed 1280 x 720, fullscreen 1920 x 1080, restore, resize to 1000 x 600, and smoke exit all worked.

3. Scope boundaries

The active renderer intentionally does not use GLSL shaders, VBOs, VAOs, image textures, a general shadow system, true reflection, or refraction. Visual richness comes from geometry, materials, lighting, fog, blending and animation. Barns are the only world structures with camera collision. These boundaries are documented as limitations, not silently described as implemented functionality.

Repository identity. The README identifies the student as Niloy Chowdhury, roll 2107117. No unsupported institutional metadata has been added.

Architecture and frame pipeline

1. Active ownership

Layer	Primary responsibility	Representative files
Entry/application	Window, GL context, callbacks, timing, fullscreen/VSync, main loop and title telemetry.	main.cpp, Application.cpp
Input/camera	Persistent/edge key state, mouse capture, first-person motion, view and projection.	Input.cpp, Camera.cpp
Scene coordinator	World state, toggles, animation, culling, ranked lights, collision and render order.	Scene.cpp, Animation.cpp
Rendering support	Immediate-mode primitives, materials and directional lighting.	Primitives.cpp, Lighting.cpp
Object modules	Farms, vegetation, houses, vehicles, people, animals, barns, village, ponds, sky, birds and substation.	src/objects/*.cpp

2. Per-frame sequence

1Poll events	→	2Apply app toggles	→	3Scene input	→	4Camera update
8Swap + telemetry	←	7World render	←	6Sky render	←	5Collision + simulation

Initialization creates a 1280 x 720 GLFW window, requests an OpenGL 2.1 context, installs callbacks, enables depth testing and linear fog, applies the initial projection and initializes input/scene state. Each loop clamps long time steps, processes controls, updates simulation values, resolves barn collision, draws the sky with camera translation removed, then draws the translated world.

3. Active versus inactive code

Status	Modules and implications
ACTIVE	Scene directly uses the object modules documented on pages 5–14, including the bridge implemented inside Pond.cpp and the active PowerSubstation.
INACTIVE	FarmWorld is never instantiated; therefore its standalone Bridge, PowerPlant and TextureManager path is not part of the rendered application.
INACTIVE	Shadow.cpp is compiled but unused. Transform.h, Cloud::drawCloud, Scene::renderTrees, Scene::renderAnimals, and the commented transformation marker are not active.
STALE	Animation::cloudOffset is updated but the active cloud field owns its own movement. The README’s older cloud-offset/phase statements should not override current source behavior.

World composition and object checklist



Figure 1a. Genuine day capture: road, farms, people, poles, trees and traffic.



Figure 1b. Genuine night capture: moon, fog, lamps and tractor headlights.

1. Placement system

`Scene::render` places 12 reusable farms in six rows beside a central road. For row $r=-2\dots3$, $z=34r$, $x=-38$ or $+38$, and farm scale is 0.78 . This is the actual placement equation: $P_{world}=(x,0,z)+0.78\ R(\text{yaw})\ P_{local}$. P_{local} is a point in one farm; R applies its small layout rotation.

2. Source-checked active inventory

Group	Distinct implemented objects and features
Terrain/road	Ground, asphalt, shoulders, dashed center line, entrance gaps, weathering patches, paths, rocks, flower/grass clearings.
Farm	Farmhouse, porch/steps/rails, doors, windows/shutters, chimney, crop soil/rows, fence/gate, trees, crops, flowers, shrubs, windmills.
Actors/vehicles	Three road tractors with drivers, manual farm tractors, four roadside farmers, 18 crop workers, cows, hens/roosters, villagers and ducks.
Village/props	Timber, brick and bamboo tea shops; counters, shelves, kettles, cups, jars, packets, signs, tables, stools, benches, bicycles, bins and plants.
Landmarks	Four barns and interiors; three ponds; active pond bridge; five bonfire sites; electrical substation; distribution poles, wires and lamps.
Atmosphere	Gradient dome, sun, moon/craters, stars, 96 clouds, day/night birds, linear fog, water ripples/sparkles, smoke, steam, sparks and light pools.

Coverage rule. Pages 6-15 give each closely related group a screenshot, code location and object-specific mathematics. Repeated instances share the representative image because their design function is identical.

Farm construction and variation



Figure 2. A populated farm cell: fence, crop rows, farmhouse, cows, workers, trees, pond edge and surrounding farms.

1. Farmhouse, field, fence and gate

`Scene::renderFarm` establishes a farm frame; `Farmhouse::render` then builds foundation, timber body, gables, two sloped roof slabs, framed windows, shutters, door, chimney, porch, steps and rails. The field is a raised soil plane with ordered `Vegetation::drawCrop` instances. Fence posts/rails leave an intentional front gate aligned with the path.

Object / code	Construction and associated mathematics
House / Farmhouse.cpp	Boxes use $p'=T R S p$. Roof slabs rotate about Z; paired signs of the angle create the two pitches. Repeated muntins/balusters use $x_i=x_0+i$ spacing.
Crop / Vegetation.cpp	A curved stalk center follows $x(y)=0.06y^2$; blades, grain heads and color-shaded cylinders are attached in the stalk frame.
Fence, gate, rocks	Posts repeat at fixed intervals; rails span post midpoints. Rock yaw/scale and grass variation come from stable hashes, so placement does not flicker.
Trees/flowers / Vegetation.cpp, Village.cpp	Tapered trunk radius is sampled by height. Blossom point $(x+r \cos a, y, z+r \sin a)$ distributes petals/flowers around stems and crowns.
Windmill / windmill.cpp	Tapered width is linear interpolation $h(y)=h_0+(h_1-h_0)t$. Rotor angle obeys $a+=\omega \Delta t$; four blades use $a+i(90 \text{ degrees})$.

2. Variation and reuse

`FarmLayout` changes tree count, zero-to-two cows, zero-to-three chickens, house/crop offsets and optional tractor/windmill. Geometry functions are reused; only parameters differ. Static field, boundary and vegetation groups are recorded in display lists, while animals, house lighting, tractor and rotor remain dynamic.

Image reading. Figure 2 shows farmhouse, fence/gate, crop rows, grass, mature trees, cows and workers together. Windmills are visible behind the substation in Figure 9a.

Road system and vehicles



Figure 3a. Road hierarchy, utility corridor, pedestrian and oncoming tractor.



Figure 3b. Reusable farm tractor beside a farmhouse and fenced plots.

1. Road, markings and weathering

`Scene::renderRoad` draws a 9-unit-wide plane, two soil shoulders and 64 deterministic flat patches. Center dashes repeat every 12 units; a dash is skipped when `|z-rowZ|<6` at each farm entrance. Small Y offsets (0.045...0.061) prevent z-fighting.

Object / code	Build, movement and equation
Manual tractor / Tractor.cpp	Chassis, engine, cabin/glass, roof, axles, wheels, fenders, lamps, exhaust, bumper, steps, seat, steering wheel and hitch share one root. With heading h, $dx=-\sin(h)d$, $dz=-\cos(h)d$, $d=5$ input Δt .
Wheels	Rolling angle follows traveled distance: $\Delta \text{Angle}=d/r * 180/\pi$, not elapsed time alone. This keeps wheel spin consistent during forward and reverse travel.
Road convoy	Three red/green/blue tractors use $z+=5.5 \Delta t$ on $[-108,108]$. Initial separation is $216/3=72$, above the configured 50-unit minimum.
Drivers/headlights	<code>Village::drawDriver</code> is seated inside each cab and transformed by the vehicle root. Headlights are spotlights whose position/direction follow that same root.
Road pedestrians	Four <code>Farmer</code> objects alternate 24-46 s active and 12-25 s waiting states; a same-side spawn must be at least 18 units clear.
Controls. <code>I/K</code> drive and <code>J/;</code> steer the manual tractor. <code>T</code> pauses convoy and roadside traffic. Vehicles are visual geometry, not camera colliders.	

Animals and agricultural activity



Figure 4a. Cows, coat patches, crop workers and field rows.



Figure 4b. Farm overview with cows and a small chicken beside the house.

1. Cow / `Animals::drawCow`

The cow hierarchy includes contact patch, barrel/chest/rump, belly/ribs, withers/hips, three deterministic coat layouts, neck/dewlap, muzzle/mouth/nostrils, eyes/blink, ears/tag, horns, collar/bell, four jointed legs, cloven hooves/dewclaws, udder and tail. `seed=0.37x+0.23z` separates timing. Grazing uses `g=clamp((0.5+0.5 sin(rate*t+phase)-0.20)/0.42)`; breathing is `1+0.012 sin(1.3t+seed)`. Tail and ear fly-swats use high powers of positive sine, producing occasional pulses rather than constant flapping.

2. Hen/rooster / `Animals::drawChicken`

Body, breast, feather layers, wings, legs/toes, beak, eye, comb/wattle and a different hen/rooster tail share one model. Wander angle is `a=a0+0.55t+(0.8*0.55/0.9)sin(0.9t+seed)`; position is `(x+1.1 sin a,z+1.1 cos a)`. Speed `1+0.8 cos(0.9t+seed)` drives walking; low speed blends into pecking.

3. Farmers and villagers

Crop workers / Farmer.cpp	18 workers (balanced 0-3 per farm) move toward row targets by normalized step $p+=((\text{target}-p)/\text{distance}) \min(\text{speed } \Delta t, \text{distance})$. Arm/leg swing is $28 \sin(\text{cycle})$.
Road farmers	The same articulated model follows a road route and state timer. Heading is $\text{atan2}(dx,dz)$.
Shop, bench, fire people / Village.cpp	Seated/standing poses use tapered limb segments and two-link arm placement. Gesture, head turn and sip motion use phase-shifted sine waves.

State. Chicken counts and worker/bench populations are randomized once at startup, then remain stable. **O** pauses workers; brackets change work speed; **P** pauses the shared animation clock.

Village life and roadside shops



Figure 5a. Timber stall.



Figure 5b. Brick veranda shop.



Figure 5c. Bamboo stall at the road edge.

1. Three distinct shop variants

Variant	Identity and contents
Timber stall	Board construction and tin-roof profile, open counter, sign, stocked shelves, cups/jars/kettle, bench/table seating, bicycle and customers.
Brick shop	Compact masonry room with a veranda, framed openings, counter activity, seated tea group and warm window/veranda lighting.
Bamboo stall	Raised bamboo floor and stilts, woven mat walls, deep thatched awning, bamboo counter, mud stove, lantern/string bulbs, bananas and seating.

Village::drawRoadsideAmenities places the three variants near the road. Static shells are cached; customers, steam, fire glow and electric lamps stay dynamic. Timber roof underside uses the code equation $y=4.24-0.80(z+2.8)/5.7$; bamboo roof center uses $y=4.9-1.2(z+2.6)/7.3$.

2. People, seating and small props

A common seat surface $Y=0.96$ and hip $Y=1.12$ align articulated people with benches, stools and split logs. Four roadside benches store 0-2 occupants in two-bit fields. Cups, trays, snack packets, kettle, biscuit jars, shelf stock, signboards, plants, bin and bicycle are separate transformed primitive groups, not textures.

3. Effects mathematics

Steam life is $\text{life}=\text{fract}(0.22t+\text{phase}+0.2i)$; alpha is $0.26(1-\text{life})\text{min}(1,6\text{life})$, while horizontal sine drift and upward translation make each puff rise and fade. Fire glow intensity is $0.70+0.20\sin(9t+x)+0.10\sin(17t)$. Electric shop lamps additionally multiply by night and grid-power amounts.

Figure mapping. 5a shows timber stock, counter, bench, villagers and bicycle; 5b shows the brick veranda/table group; 5c shows the bamboo stall on the right of the road.

Barn system: architecture and interior



Figure 6. Accessible barn entrance showing sliding door, wheelbarrow, aisle, stalls, hay, trough and tools.

1. Shell / Barn::draw

Four transformed instances reuse a 10 by 8 local footprint. Foundation, separate red wall panels/battens, gables, windows, side/hayloft doors and two 30-degree roof slabs leave a genuine front opening. Ridge height is $5.5+(5.0+0.5)\tan(30\text{ degrees})$; underside height at local X follows $\text{ridge}-|x|\tan(30\text{ degrees})$. Posts, tie beams and rafters use endpoint midpoint, length and atan2 orientation.

2. Interior object inventory

Circulation	Clear central aisle, doorway threshold, sliding track and collision-aware opening.
Animal areas	Side stalls, rails, dividers, feeding troughs and a hinged feeding gate.
Storage	Hay bales/stacks and loft, shelving/items, sacks, ladder, hinged chest, workbench, tack wall and hand tools.
Equipment	Moveable wheelbarrow/rotating wheel, bucket, stool, milk can and fenced outside area.
Lighting	Exterior fixture and interior pendant lights whose emissive output and local-light contribution follow grid power.

The cached static list holds unchanged architecture/content; sliding doors, chest lid, feeding gate, wheelbarrow and lamps render separately. Door translation is proportional to normalized open amount. Chest/gate apply hinge rotation $\text{angle}=\text{maxAngle}*\text{openAmount}$.

3. Instance state

All four barns currently share the same mechanism values in Scene. One key therefore moves every corresponding prop. Figure 6 is a genuine open-door capture and clearly shows aisle, stalls, trough, bales, shelf/tools and wheelbarrow.

Barn interaction, collision and power



Figure 7a. Night, grid power on: exterior and interior warm illumination.



Figure 7b. Same view after load shedding: electric fixtures and hay illumination fade.

1. Interaction controls

G	Toggle all sliding barn doors. Door amount approaches the target smoothly at about 0.58 units/s.
Z/X	Hold to move the wheelbarrow toward the entrance/rear. Travel is clamped to approximately -1.35...+1.10 and wheel spin follows distance.
C	Toggle chest lid; a bounded scalar drives hinge rotation at about 1.25 units/s.
F	Toggle feeding gate; its hinge parameter approaches the target at about 1.05 units/s.
L	Toggle load shedding. A smoothed powerAmount fades grid-dependent lights rather than switching them discontinuously.

2. Door-aware camera collision

The scene transforms camera motion into each barn’s local coordinates, samples a long movement in smaller substeps, and rejects samples that intersect wall bands. The front wall is treated specially: its opening half-width grows with door animation, capped near 3.35 units. A closed door blocks the center; a sufficiently open door admits the camera. The resolved local point is transformed back to world space.

Substepping uses $steps = \text{ceil}(\text{travel} / (0.14 * \text{barnScale}))$, avoiding tunneling. For barn world offset (wx, wz) , rotation r and scale s , local position is $lx = (\cos(r)wx - \sin(r)wz) / s$, $lz = (\sin(r)wx + \cos(r)wz) / s$. Wall-band tests run in this reusable local frame.

3. Independent and dependent energy

Barn bulbs are grid-dependent. The power comparison above shows their emissive surfaces, interior visibility and local-light intensity fading together during an outage. Manual door/prop interactions remain available because they are not modeled as electrical actuators. Bonfires likewise remain lit; their flames and point lights are intentionally independent of the grid.

Current limitation. Collision is implemented for barns only. Fences, trees, vehicles, shops, water and substation geometry remain non-solid to the camera.

Ponds, animated water and bridge



Figure 8a. Rural wooden bridge, layered bank and forest setting.



Figure 8b. Water surface, duck, lily pads, reeds, stones and circular ripples.

1. Pond composition

Three ponds break up the repeated farm grid. Each combines an irregular-looking layered soil/sand perimeter with water, stones, reeds and grass. The main and secondary sizes are parameterized (about 36×26 and 32×23 world units), while placement and surrounding details create distinct landmarks.

The water is an eight-ring, 48-segment translucent polar mesh. At angle a and normalized radius s , $x = r_x * \text{edge}(a) * s * \cos(a)$, $z = r_z * \text{edge}(a) * s * \sin(a)$. Height/color shimmer uses $w1 = \sin(1.6t + 1.4x + 1.1z)$ and $w2 = \sin(2.3t - 0.9x + 1.7z)$. Ripples expand from $\text{radius} = 0.05 + \text{phase} * 0.14 * \min(r_x, r_z)$ while alpha falls as $0.55(1 - \text{phase})$.

2. Bridge construction

The active bridge in `Pond.cpp` includes piles, side beams, 13 deck planks, posts and two sampled rails per side. Contrary to the earlier draft, it does **not** use a cubic Bezier. Its actual arch is:

$$z(t) = (2t - 1)(0.47L), y(t) = \text{base} + \text{rise} * \sin(\pi * t), 0 \leq t \leq 1$$

Twelve neighboring samples become boxes. Midpoint sets translation, $\sqrt{(\Delta z)^2 + (\Delta y)^2}$ sets length, and $-\text{atan2}(\Delta y, \Delta z)$ sets X rotation. Ducks use elliptical paths ($r_x * 0.42 \cos a, r_z * 0.40 \sin a$); velocity supplies heading. Lily pads bob by $0.004 \sin(1.2t + \text{seed})$.

3. Render-state discipline

Opaque banks, rocks and bridge render with normal depth writes. The water/effects pass enables alpha blending and restores state afterward. Ripples sit slightly above the surface to avoid z-fighting. This ordering is important because legacy transparency does not automatically sort fragments.

Module distinction. The standalone `Bridge.cpp` belongs to the inactive `FarmWorld` path. The bridge visible in Figures 8a–b is the independently implemented active bridge in `Pond.cpp`.

Electrical infrastructure and load shedding



Figure 9a. Fenced substation building, warning signs, conductors and equipment.



Figure 9b. Powered night scene with road lamps and vehicle lighting.

1. Substation / **PowerSubstation.cpp**

The active fenced yard contains slab/gravel, operations building, roof equipment, door/windows/louvres, warning signs, stack, two transformers, fire walls, gantry, current/voltage transformers, surge arresters, bus and cable equipment, floodlights, generator, drums, kiosk, storage and earth straps. Gate mesh, tubes and beams are generated geometrically. A segment from A to B uses midpoint $(A+B)/2$, length $|B-A|$ and an axis-angle alignment.

2. Distribution corridor

Utility poles repeat down both road sides. Crossarms and insulators support multiple conductors. Instead of straight lines, each wire span samples a parabolic sag term:

$$y(t) = (1-t)y_0 + ty_1 - 4s\,t(1-t)$$

The factor $4t(1-t)$ is zero at each pole and one at the midpoint, giving controllable maximum sag s . This inexpensive curve strongly improves depth and realism.

3. Dynamic equipment and load shedding

Roof fans rotate from animation time, beacons use $\text{phase}=\text{fmod}(t,1.5)$, and six stack-smoke puffs use $\text{life}=\text{fract}(\text{raw})$, $\text{radius}=0.16+0.42\text{life}$, $\text{alpha}=0.70(1-\text{life})$. L changes a target power state; $\text{blend}=1-\exp(-5\,\text{Delta}T)$ and $\text{power}+=(\text{target-power})\text{blend}$ smoothly fade electrical output.

Road lamps, shop strings/lanterns where electrically modeled, farmhouse/barn fixtures and substation indicators scale with this value. Bonfires remain independent. The scene also applies restrained nighttime outage darkening to terrain so the visual story remains coherent without making the world unreadable.

Inactive alternative. `PowerPlant.cpp` is compiled but not instantiated by the active `Scene`. It must not be confused with the visible `PowerSubstation`.

Sky, weather and environmental effects



Figure 10a. Viewer-facing sun and cloud field.



Figure 10b. Layered flame, stone ring and log seating.



Figure 10c. Moon, night fog and local illumination.

1. Sky and celestial bodies

The sky is drawn with camera rotation but without translation, so it appears infinitely distant. Day color, night color and linear fog interpolate from the same day/night amount. The sun is a camera-facing disk with three restrained glow layers. It uses the same `DayNightSettings::SunPosition` as the directional light, preventing a common mismatch between visible sun and shading direction. Depth writes are disabled for the translucent glow.

At night, a moon disk/crater treatment and deterministic star field fade in. `N` changes the target; `night=clamp(night +/- DeltaT/3,0,1)`. The sky view uses camera rotation but omits translation, so the dome/celestial bodies cannot be walked past.

2. Clouds and birds

`cloud.cpp` distributes 96 clouds uniformly over an annulus: $r = \sqrt{r_{min}^2 + u(r_{max}^2 - r_{min}^2)}$, $x = r \cos a$, $z = r \sin a$. Each moves by $p += direction * speed * DeltaT$ and wraps to the opposite rim. Day birds use `travel=fract(t/period+phase)`, V formations and `wing=32sin((5+0.35f)t+phase)`; a sparse night pair crosses the moon during 38% of a 22-second cycle.

3. Bonfire system

Five clearings are defined at fixed world positions. Each site contains a stone ring, crossed round logs, ash/embers, four layered drifting flame forms, two smoke puffs, five sparks and nearby villagers seated on split logs. `B` toggles all sites. At night, nearby active sites can contribute warm, flickering point lights to the ranked local-light set; their illumination is not tied to load shedding.

Smoke/spark life is `fmod(rate*t+phase,1)`. Smoke rises to $y=1.55+2.4life$, expands and fades; sparks rise to $1+2life$ with sine/cosine drift. Flame and selected local-light intensity use two sine frequencies. Villagers reuse the seated/standing human model. `B` toggles all sites.

Fog/transparency. Linear fog blends distant trees toward the interpolated sky color. Clouds, glow, smoke, flames, water and light pools use alpha blending with explicit depth/state restoration.

Rendering, materials and lighting

1. Rendering pipeline

The application requests OpenGL 2.1 and uses the legacy compatibility pipeline: immediate-mode vertex submission, matrix stacks, fixed-function materials, `GL_LIGHTING`, fog and display lists. There are no GLSL programs, vertex/index buffers or vertex array objects. This makes the transformation and lighting model explicit for study, although it limits modern GPU efficiency.

2. Opaque and transparent passes

Stage	State and purpose
Frame setup	Clear color/depth buffers; apply projection and view; configure fog and global light state.
Sky	Camera translation removed; celestial glow uses blending and disabled depth writes where required.
Opaque world	Depth test/write on; ground, road, buildings, farms, vegetation, props, characters and infrastructure.
Local illumination	Select and configure up to seven additional point/spot lights around the camera.
Transparent effects	Water, smoke, cloud layers, particles and projected light pools use source-alpha blending; state is restored.

3. Lighting equations applied to objects

`GL_LIGHT0` is directional ($w=0$). Direction and ambient/diffuse/specular colors use $C=(1-n)C_{day}+nC_{night}$, where n is night amount. Fixed-function shading is conceptually $I=K_aL_a+K_dL_d\max(N\cdot L,0)+K_sL_s\max(R\cdot V,0)^{shininess}$. N is the supplied surface normal; `GL_NORMALIZE` repairs its length after scaled cows, trees, vehicles and buildings.

Point/spot contribution falls by $1/(1+k_L*d+k_Q*d^2)$. Candidate farmhouse, shop, barn, road-lamp, bonfire and headlight sources are ranked by priority then squared camera distance; at most seven occupy `GL_LIGHT1...7`. Headlights also use spot direction/cutoff/exponent.

4. Visual approximations

Bulbs/flames	Emissive material makes a source look bright; selected sources also illuminate geometry.
Light pools	Translucent ground geometry suggests spill beyond the small hardware-light budget.
Water	Alpha, brightness variation and ripple geometry; no reflection/refraction render targets.
Shadows	No active general shadow pass. Cows include a simple flattened dark contact shape; other depth cues come from lighting, overlap and fog.
Textures	No active texture assets; surface identity is built from color/material and additional geometry.

Why the approach works. The scene invests in silhouette, hierarchy, color contrast and secondary motion. Those choices produce recognisable objects even without textures or programmable shading.

Transformation, camera and projection mathematics

1. Model transformations

Vertices begin in local object coordinates. A typical hierarchy uses translation **T**, rotation **R** and scale **S**. With OpenGL's post-multiplied matrix calls, a vertex is effectively evaluated as:

$$p_{\text{world}} = T \cdot R \cdot S \cdot p_{\text{local}}$$

The practical order is therefore scale the primitive, rotate it into orientation, then place it in the parent/world. Nested `glPushMatrix/glPopMatrix` pairs establish a local coordinate frame for a farm, tractor, cow, villager, barn or shop and safely return to the parent afterward.

2. Camera basis

Let yaw be ψ and pitch be θ , converted to radians. The camera derives:

$$\begin{aligned} \text{forward} &= (\cos\psi \cos\theta, \sin\theta, \sin\psi \cos\theta) \\ \text{right} &= (-\sin\psi, 0, \cos\psi) \\ \text{up} &= \text{normalize}(\text{right} \times \text{forward}) \end{aligned}$$

The view matrix places the orthonormal basis in the current matrix and translates by the negative camera position. The sky view uses the same basis but omits translation. Keyboard walking intentionally uses only the horizontal heading ($\cos\psi, \sin\psi$); looking upward never causes W/S to fly.

Input axes are summed and the combined direction is normalized only when its length exceeds one. Thus diagonal vertical/forward movement is not faster. Position integration is direct:

$$\text{position}_{\text{new}} = \text{position}_{\text{old}} + \text{direction} \cdot \text{speed} \cdot \Delta t$$

Base speed is 20 units/s; Left Shift multiplies by 3 and Left Ctrl by 0.3. Yaw changes at 90°/s from A/D or arrows, mouse sensitivity is 0.08°/pixel, pitch is clamped to $\pm 89^\circ$, and yaw wraps into $[-180^\circ, 180^\circ]$. Frame time is capped at 0.25 s to contain focus/load spikes.

3. Perspective projection

For vertical field of view $f = 45^\circ$, aspect $a = \text{width}/\text{height}$, near distance $n = 0.1$ and far distance $F = 1000$:

$$\text{top} = n \cdot \tan(f/2), \text{right} = \text{top} \cdot a; \text{glFrustum}(-\text{right}, \text{right}, -\text{top}, \text{top}, n, F)$$

The framebuffer-size callback recomputes this projection after resize/fullscreen changes. A nonzero height guard prevents division by zero.

4. Frustum and distance culling

The scene multiplies projection and model-view matrices, extracts six planes using row 4 $\pm$ row 1/2/3, then normalizes them. A bounding sphere is outside when $\text{plane} \cdot \text{center} + d < -\text{radius}$. Squared distance tests avoid square roots for broad culling. Combined with fog, this prevents large off-screen/distant groups from consuming full draw cost.

Simulation, curve and collision mathematics

1. Frame-rate-independent animation

Most linear state follows $x \leftarrow x + v\Delta t$ and is clamped or wrapped. Periodic motion uses $\sin(\omega t + \phi)$, where a deterministic phase ϕ desynchronizes instances. Smooth target following uses:

$$\alpha = 1 - e^{-k\Delta t}, x \leftarrow x + (\text{target} - x)\alpha$$

This exponential form is used for power and response values because it remains stable across frame rates. Day/night uses a bounded linear progression of about $\Delta t/3$. Global pause stops the shared animation progression; traffic and workers additionally expose subsystem pauses.

2. Vehicle and rotational motion

A steered vehicle advances along its heading h with $(\Delta x, \Delta z) = d(\cos h, \sin h)$. Wheel angle is tied to traveled distance, conceptually $\Delta \text{angle} = 180d/(\pi r)$ degrees. Barn wheelbarrow code uses an equivalent calibrated factor near 150.78° per travel unit. Road vehicles add $\text{speed} \times \Delta t$ on their route and wrap only while preserving safe spacing.

3. Procedural distribution

Stable integer/hash seeds derived from chunk, farm or object coordinates choose placement jitter, scale, cow coat layout, timing phase and vegetation. Startup RNG chooses chicken counts, balanced worker populations and bench occupancy once. World grass is generated in 40-unit chunks and rejected by analytical road, path, building, pond and fire-clearance regions.

4. Curves and particles

Active bridge rail	$z(t) = (2t-1)0.47L$, $y(t) = \text{base} + \text{rise} \sin(\pi t)$; samples form oriented boxes. The cubic Bezier in standalone Bridge.cpp belongs to inactive FarmWorld.
Wire sag	Linear endpoint interpolation minus $4s \, t(1-t)$.
Particle life	$\text{life} = \text{fract}(\text{rate} \times \text{time} + \text{phase})$; smoke rises/expands and alpha falls with life; sparks combine upward life with sine drift.
Viewer-facing disks	Sun/moon geometry is rotated toward the current view so a thin disk remains legible.

5. Barn collision transform

For a barn translated/rotated/scaled in world space, camera offset is inverse-rotated and divided by scale:

$$l_x = (\cos r \cdot w_x - \sin r \cdot w_z)/s, \quad l_z = (\sin r \cdot w_x + \cos r \cdot w_z)/s$$

Local wall bands and the animated opening can then be tested with simple comparisons. The accepted point is rotated/scaled back. This is cheaper and clearer than testing world-space triangles and naturally reuses one collider for every transformed barn.

Numerical care. Capping Δt , clamping bounded states, wrapping angles/routes, normalizing vectors/planes, and subdividing long collision motion prevent common instability and tunneling problems.

Complete control reference

Input	Action	Scope / behavior
W / S	Move forward / backward	Horizontal camera heading; frame-time integrated.
A / D	Yaw left / right	Rotates in place; can be combined with W/S.
Q / E	Move up / down	Vertical camera translation.
Left Shift	Sprint	3.0× camera speed while held.
Left Ctrl	Precision move	0.3× camera speed while held.
Mouse / click / Tab	Look / capture cursor	Click or Tab captures; mouse changes yaw/pitch. Tab toggles.
Arrow keys	Keyboard look	Left/right yaw; up/down pitch.
R	Reset camera	Returns to (0,4,10), yaw −90°, pitch −15°.
I / K	Tractor forward / reverse	Manual farm tractor.
J / ;	Tractor steer left / right	Manual tractor heading.
N	Toggle day / night	Smooth scene-wide transition.
T	Pause / resume traffic	Only the road tractor convoy.
B	Toggle bonfires	All five sites, effects and eligible local lights.
G	Open / close barn doors	Shared by all four barns; collision follows door amount.
Z / X	Move wheelbarrow	Held input toward entrance / rear, bounded travel.
C	Toggle barn chest	Shared lid state.
F	Toggle feeding gate	Shared hinge state.
L	Toggle load shedding	Fades grid-dependent illumination.
O	Pause / resume crop workers	Independent field-work control.
[/]	Decrease / increase work speed	Target constrained to 0.35...2.50 in 0.25 steps.
P	Pause / resume all animation	Global animation state.
− / +	Windmill speed down / up	Adjusts rotor animation response.
V	Toggle VSync	Updates GLFW swap interval and title status.
F11	Toggle fullscreen	Restores prior windowed size/position.
Esc	Release / exit	First press releases captured mouse; next closes.

Interaction matrix

System	Global pause P	Power L	Day/night N	Independent switch
Traffic	Paused	Headlight intensity context	Lights become visible	T
Crop workers	Paused	No direct dependency	Lighting only	O, []
Bonfires	Effect animation pauses	Independent	Light most visible at night	B
Barn props	State approach follows scene update	Mechanisms remain usable	Visibility changes	G Z X C F

Performance, testing and limitations

1. Implemented optimization

Technique	Where it helps
Display lists	Cache ground/road, farms, vegetation, shop shells, drivers, cloud forms, barns and substation static geometry.
Frustum + distance culling	Rejects farms, 225-unit trees, 90-unit world grass, 155-unit fires and other groups before detailed drawing.
Level of detail	Simplifies distant trees, drivers and clouds.
Chunked procedural detail	Generates/culls world grass locally while preserving exclusion regions.
Local-light ranking	Chooses the best seven candidates rather than enabling every visible source.
Timing safeguards	0.25 s Δt cap, VSync option, and event waiting while minimized.

2. Measured runtime and validation

A final 10-second instrumented run in the available review environment, after a five-second warmup with VSync disabled, reported **6.42 FPS / 155.852 ms per frame**. The workload is CPU-heavy because it submits large volumes of immediate-mode geometry; the value is environment-specific and is not a universal hardware benchmark.

The Release build succeeded. **CameraControlTests** passed 1/1 and covers heading-relative movement, opposing-key cancellation, yaw wrapping, immediate stopping, simultaneous curved motion and mouse/keyboard yaw continuity. It does not test drawing, scene toggles, collision or visual output. Manual smoke validation covered startup, windowed/fullscreen/restore/resize, day/night, power, barn interaction and clean exit.

3. Known limitations and maintenance notes

- Legacy immediate mode and display lists limit performance and portability relative to modern buffered rendering.
- No active shaders, texture mapping, general shadow maps, reflections, refraction, skeletal animation or physics engine.
- Camera collision applies only to barns; all other geometry can be traversed.
- All barn instances share one interaction state rather than per-barn state.
- Startup randomization means worker/chicken/bench occupancy varies between launches, though it remains stable within one run.
- FarmWorld, standalone bridge/power plant/texture manager, Shadow, and several helper paths are inactive or dead code.
- farmworld.cpp includes FarmWorld.h while the repository filename is farmworld.h; this is a case-sensitive-platform hazard.
- .gitignore matches test*; the present tests/CameraControlTests.cpp is locally available and referenced by CMake, but clean-clone tracking should be checked.
- README contains stale phase and cloud-offset statements; current code is authoritative.

4. Recommended next steps

Highest-value improvements are modern VBO/VAO + shader rendering, batched/instanced repeated geometry, per-barn state, broader collision, transparent-object sorting, expanded automated scene-state tests, and cleanup of inactive modules/case-sensitive includes. A profiling pass should precede aesthetic expansion.

Conclusion, viva notes and references

1. Conclusion

The current 3D_farm_OpenGL repository demonstrates a complete interactive scene rather than a collection of disconnected models. Its strongest achievement is integration: reusable primitive-based objects operate inside one navigable world; transforms support twelve varied farm instances; animation connects animals, people, vehicles, water and weather; day/night and power states affect a shared lighting model; and targeted collision makes the barn a genuinely enterable environment.

The project also exposes important engineering tradeoffs. Fixed-function OpenGL makes matrices, lights and materials easy to inspect, but the high immediate-mode submission cost is visible in the benchmark. Stylized geometric effects keep dependencies small, but cannot supply the realism of textured shader pipelines. Documenting those facts strengthens the project: it shows a clear understanding of what was implemented, how it works, and what should change next.

2. Short viva / demonstration guide

1. Begin at the road and explain the Application → Scene → object modules frame path.
2. Use W/S, A/D and mouse look; derive the forward vector and show movement is frame-time based.
3. Toggle N and identify the shared sun direction, sky/fog interpolation and local night lights.
4. Visit a barn; use G, enter through the animated collider, then demonstrate Z/X, C and F.
5. Toggle L at night and contrast powered fixtures with grid-independent bonfires.
6. Show traffic/work subsystem controls and explain display lists, frustum/distance culling, LOD and the seven-light ranking.
7. Close by distinguishing active Pond bridge and PowerSubstation from compiled-but-unused legacy modules.

3. Repository references (source of truth)

[1]	CMakeLists.txt; build targets, C++ standard, GLFW/OpenGL linkage and test registration.
[2]	src/main.cpp, src/Application.cpp, src/Input.cpp; lifecycle, callbacks, window modes and input state.
[3]	src/Camera.cpp, src/Scene.cpp, src/Animation.cpp, src/Lighting.cpp; camera mathematics, orchestration, simulation and lighting.
[4]	src/graphics/Primitives.cpp; reusable compatibility-mode drawing primitives.
[5]	src/objects/Animals.cpp, Farmer.cpp, Tractor.cpp, Vegetation.cpp, Farmhouse.cpp, windmill.cpp; farm actors and objects.
[6]	src/objects/Barn.cpp, Village.cpp, Pond.cpp, PowerSubstation.cpp; major interactive landmarks and amenities.
[7]	src/objects/sky.cpp, cloud.cpp, Birds.cpp; sky and atmospheric content.
[8]	include/DayNightSettings.h, include/VillageSimulationSettings.h; shared constants and simulation parameters.
[9]	tests/CameraControlTests.cpp; automated camera control coverage.
[10]	README.md, LICENSE; project identity, historical description and MIT licensing.

END OF REPORT · EXACTLY 20 PAGES · SOURCE-VERIFIED